PROGRAMACIÓN 1 · LABORATORIO 2026

Cómo afrontar el Laboratorio de Programación 1 (2026)

Guía de abordaje

Del enunciado al programa en Python orientado a objetos

Noviembre de 2026

Índice

1 Cómo usar esta guía y qué se entrega ... 4
1.1 Cómo leerla ... 4
1.2 Qué se entrega ... 4
2 Entender el problema antes de programar ... 4
2.1 El dominio en una frase ... 4
2.2 Actores y conceptos ... 5
2.3 Reglas de negocio extraídas de la letra ... 5
2.4 Fórmulas ... 6
2.5 Dos ejemplos calculados ... 6
3 Del enunciado al diseño ... 7
3.1 El diagrama oficial ... 7
3.2 Clase por clase ... 7
3.3 Por qué OpcionIA es abstracta ... 8
3.4 Por qué Evaluacion guarda referencias ... 8
3.5 Sistema, el coordinador ... 8
3.6 El diccionario de puntajes ... 9
3.7 La interfaz pública de Sistema ... 9
3.8 Una operación de punta a punta ... 9
4 Decisiones de arquitectura ... 10
4.1 Dominio sin entrada ni salida ... 10
4.2 Tabla de decisiones ... 10
4.3 La jerarquía de excepciones ... 11
4.4 Calcular bajo demanda o guardar ... 11
4.5 Encapsulamiento en la práctica ... 11
5 Estructura del proyecto y convenciones ... 12
5.1 Árbol de carpetas ... 12
5.2 Imports ... 12
5.3 Cómo ejecutar ... 12
5.4 Mapa de menús ... 13
5.5 Qué muestra cada listado ... 13
5.6 Convenciones de código ... 13
5.7 Formato de números y fechas ... 14
6 Plan de construcción en 7 etapas ... 14
6.1 Las siete etapas ... 14
6.2 Qué reglas cubre cada etapa ... 16
7 Implementación comentada ... 16
7.1 Excepciones propias ... 16
7.2 Criterio y CasoUso ... 16
7.3 OpcionIA, ServicioAPI y PlanMensual (abstracción y polimorfismo) ... 17
7.4 Evaluacion y el cálculo del estado ... 18
7.5 Sistema: registrar y validar ... 18
7.6 Sistema: evaluar y comparar ... 19
7.7 Sistema: actualizar y recalcular ... 20
7.8 Sistema: ranking ... 20
7.9 main.py: lectura robusta y menús ... 21
8 Validaciones y manejo de errores ... 23
8.1 Una capa por regla ... 23
8.2 Catálogo de validaciones ... 24
9 Casos borde y errores frecuentes ... 25
9.1 El costo igual al presupuesto ... 25
9.2 Un ejemplo de 6.996 ... 25
9.3 Empates en el ranking ... 26
9.4 Limitaciones conocidas ... 26
10 Cómo probar ... 27
10.1 Guion de prueba manual ... 27
11 Entrega y checklist final ... 29
11.1 README ... 29
11.2 Archivo comprimido ... 29
11.3 Checklist final ... 29
12 Lista de tareas ... 30
12.1 Etapa 0: preparación ... 30
12.2 Etapa 1: clases del diagrama y excepciones ... 30
12.3 Etapa 2: menú principal y submenús ... 30
12.4 Etapa 3: operaciones de Registrar ... 30
12.5 Etapa 4: operaciones de Listar ... 31

12.6 Etapa 5: evaluación individual 31
12.7 Etapa 6: comparar opciones y ranking 31
12.8 Etapa 7: actualizar 31
12.9 Etapa 8: prueba completa y entrega 31

1 Cómo usar esta guía y qué se entrega

Esta guía acompaña el Laboratorio 2026 de Programación 1: un programa de consola en Python, orientado a objetos, que compara opciones de inteligencia artificial para los casos de uso de una empresa.

La guía no reemplaza a la letra. La letra define qué debe hacer el programa. La guía muestra cómo pensarlo, diseñarlo, construirlo y probarlo, y explica por qué se toma cada decisión. Si alguna vez ambas parecen contradecirse, manda la letra.

1.1 Cómo leerla

Conviene leer los capítulos en orden la primera vez y usarlos después como consulta.

Capítulos	Para qué sirven
2 y 3	Entender el problema y traducirlo a clases.
4 y 5	Conocer las decisiones de arquitectura y la estructura del proyecto.
6 y 7	Construir el programa por etapas y estudiar la implementación comentada.
8 y 9	Validar datos y atender los casos borde.
10 y 11	Probar el programa y preparar la entrega.

A lo largo del texto aparecen tres tipos de recuadro:

> **Decisión**
> Explica una decisión de diseño y el motivo por el que se tomó.

> **Atención**
> Advierte sobre una trampa frecuente o un error difícil de detectar.

> **Checkpoint**
> Indica qué debe poder comprobarse por consola antes de seguir avanzando.

1.2 Qué se entrega

La siguiente tabla resume lo que pide la letra en sus «Consideraciones generales» (pág. 5).

Elemento	Qué pide la letra
Programa	Aplicación de consola en Python, versión `>= 3.10`.
Código fuente	Carpeta `laboratorio_2026_grupo[numero]/` con la estructura indicada en la pág. 5.
README	Instrucciones para ejecutar el programa y número de equipo.
Formato	Un archivo comprimido con el código fuente y el README, que se envía por mail.
Fecha límite	8 de noviembre, 23:59.
Librerías externas	Si se usa alguna, debe declararse y justificarse en el README. Esta guía no usa ninguna.

Las diapositivas «Tips Laboratorio 2026 – Parte 1» son la segunda fuente de esta guía. De ellas salen el orden de construcción, el patrón de menús con `while`, la lectura repetida de datos y los atributos privados de `Sistema`.

2 Entender el problema antes de programar

Antes de escribir una clase hay que poder explicar el problema sin hablar de Python. Este capítulo separa lo que dice la letra en conceptos, reglas y fórmulas.

2.1 El dominio en una frase

El **dominio** es la parte del mundo real que el programa representa. Aquí son las opciones de IA, los criterios y los casos de uso.

Una empresa registra opciones de IA, las puntúa según criterios con distinto peso y decide, para cada caso de uso, cuáles entran en el presupuesto y alcanzan el puntaje mínimo.

El programa no se conecta con ningún servicio. Claude, OpenAI o Gemini son solo nombres de ejemplo que el usuario escribe por teclado. La letra lo aclara: no se consultan servicios externos ni se usan API reales.

También conviene tener claro lo que queda fuera. La letra no pide guardar datos en archivos ni conectarse a internet. Sus menús tampoco incluyen operaciones para eliminar o editar criterios, opciones o casos: lo único que se actualiza es el presupuesto de un caso y el precio o consumo de una opción. No hace falta construir nada más.

2.2 Actores y conceptos

Hay un único actor: la persona que opera los menús de la consola. Todo lo demás son conceptos del dominio, y casi todos se convertirán en clases.

Concepto	Qué representa	Ejemplo
Criterio	Un aspecto que se evalúa, con un peso entero positivo.	Precisión, peso 3.
Opción de IA	Un servicio que puede contratarse. Tiene un puntaje por criterio y una forma propia de calcular su costo.	Servicio API o plan mensual.
Caso de uso	Una necesidad concreta de la empresa, con presupuesto mensual y puntaje mínimo.	Atención al cliente, USD 250, mínimo 7.
Evaluación	El resultado de analizar una opción para un caso, en una fecha determinada.	Claude para Atención al cliente: Recomendada.
Ranking	Las evaluaciones recomendadas de un caso, ordenadas.	Primero la de mayor puntaje.

2.3 Reglas de negocio extraídas de la letra

Cada regla de esta tabla termina siendo una validación, un cálculo o una prueba. La referencia indica la página y la sección de la letra.

Nº	Regla	Referencia
R1	El nombre del criterio es único, sin distinguir mayúsculas.	Pág. 3, Criterio
R2	El peso es un entero positivo. Los pesos no necesitan sumar 100.	Pág. 3, Criterio
R3	Todos los criterios se registran antes de la primera opción. Después no se admiten nuevos.	Pág. 3, Criterio
R4	Debe existir al menos un criterio para registrar opciones.	Pág. 3, Criterio
R5	La opción tiene nombre único y es de tipo Servicio API o Plan mensual.	Pág. 3, Opción de IA
R6	Cada criterio recibe un puntaje entero de 0 a 10, sin omisiones ni duplicados.	Pág. 3, Opción de IA
R7	Ambos tipos implementan el mismo método `costo_estimado()`.	Pág. 3, Opción de IA
R8	El caso tiene nombre único, presupuesto mayor o igual que cero y mínimo entre 0 y 10.	Pág. 3, Caso de uso
R9	Registrar un caso no genera evaluaciones.	Pág. 3, Caso de uso
R10	Si ya existe una evaluación para esa opción y ese caso, se informa y no se crea otra.	Pág. 4, Evaluación individual
R11	Cada evaluación recibe código consecutivo y fecha y hora actuales.	Pág. 4, Evaluación individual
R12	Un costo igual al presupuesto y un puntaje igual al mínimo son válidos.	Pág. 4, Evaluación individual
R13	Comparar recorre todas las opciones. Si no hay opciones, muestra un mensaje y vuelve al menú.	Pág. 4, Comparar opciones
R14	Comparar reemplaza todas las evaluaciones del caso, incluso las individuales. Las de otros casos se conservan.	Pág. 4, Comparar opciones
R15	Los códigos no se reutilizan.	Pág. 4, Comparar opciones
R16	Actualizar el presupuesto recalcula el estado de las evaluaciones del caso, sin crear evaluaciones ni cambiar fechas.	Pág. 4, Presupuesto de un caso
R17	Actualizar una opción exige costos no negativos y cantidades enteras positivas.	Pág. 4, Precio o consumo
R18	Esa actualización recalcula costo y estado de la opción en todos los casos. No cambian códigos, fechas ni puntajes.	Pág. 4, Precio o consumo

Nº	Regla	Referencia
R19	El listado de casos muestra las recomendadas actuales. Sin evaluaciones, la cantidad es cero.	Pág. 4, Listar datos
R20	El ranking incluye solo evaluaciones recomendadas: mayor puntaje, luego menor costo, luego menor código de opción.	Pág. 5, Listar datos
R21	Ante un error se pide de nuevo el dato, sin finalizar el programa.	Pág. 5, Consideraciones
R22	En cada submenú se permanece hasta elegir Salir. Una entrada incorrecta muestra un mensaje.	Pág. 3, Menú principal

2.4 Fórmulas

El costo mensual depende del tipo de opción:

$$\text{Costo API} = \frac{\text{consultas mensuales}}{1000} \times \text{precio por cada 1000 consultas}$$

$$\text{Costo plan} = \text{usuarios} \times \text{costo por usuario}$$

El puntaje total es un promedio ponderado. Un criterio con más peso influye más en el resultado:

$$\text{Puntaje total} = \frac{\sum \text{puntaje} \times \text{peso}}{\sum \text{peso}}$$

El estado se determina en este orden exacto. La primera condición que se cumple decide el resultado:

1. «Fuera de presupuesto», si el costo mensual supera el presupuesto del caso.
2. «Puntaje insuficiente», si cumple el presupuesto, pero el puntaje total es menor que el mínimo.
3. «Recomendada», si cumple ambos límites.

El orden importa. Una opción que supera el presupuesto y además tiene poco puntaje es «Fuera de presupuesto», nunca «Puntaje insuficiente».

2.5 Dos ejemplos calculados

Costo de un Servicio API (ejemplo de la letra). El precio es USD 40 por cada 1000 consultas y la empresa estima 5000 consultas al mes:

$$\frac{5000}{1000} \times 40 = 5 \times 40 = \text{USD } 200$$

Puntaje ponderado (ejemplo propio). Se definen tres criterios y una opción con estos puntajes:

Criterio	Puntaje	Peso	Puntaje × peso
Precisión	8	3	24
Velocidad	7	2	14
Soporte	9	1	9
Total		**6**	**47**

El resultado es $\frac{47}{6} = 7.8333\ldots$, que se muestra como `7.83`. Si el mínimo del caso es 7 y el costo entra en el presupuesto, el estado es «Recomendada».

3 Del enunciado al diseño

El capítulo anterior describió el problema. Este muestra cómo el diagrama oficial lo organiza en clases y por qué cada una existe.

3.1 El diagrama oficial

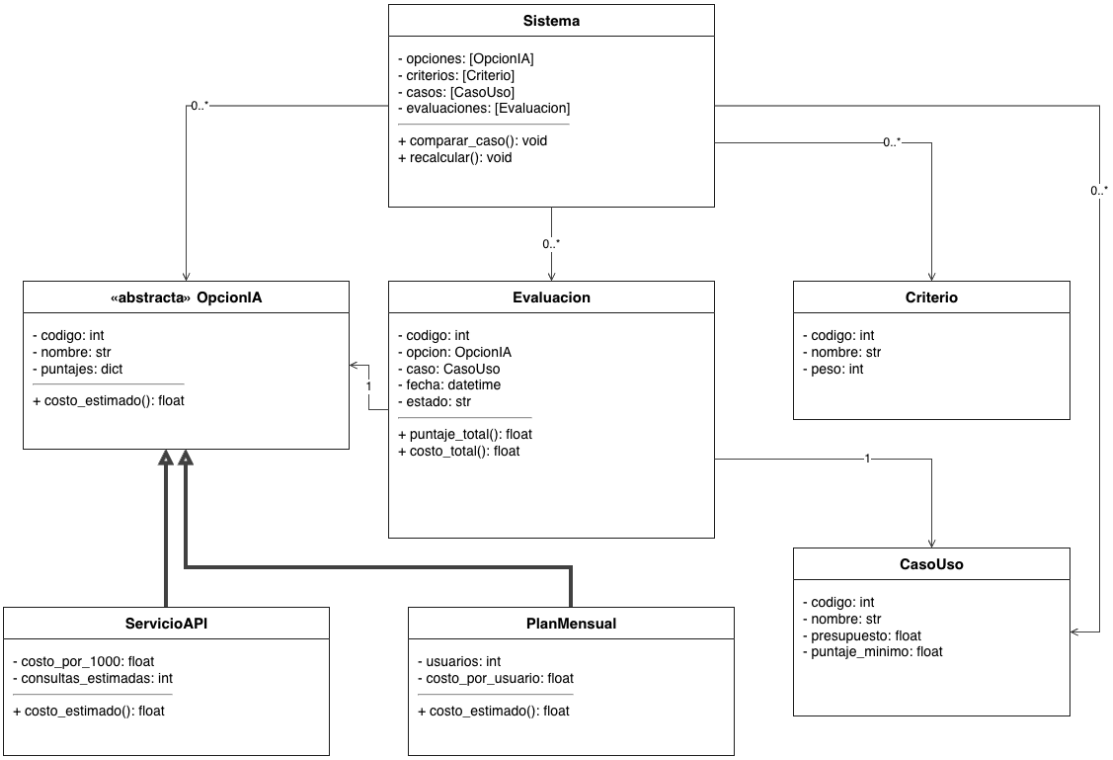


Figura 1: Diagrama de clases de la letra (pág. 2).

Para leerlo alcanza con cuatro convenciones:

- El signo `-` marca un atributo privado y el signo `+`, un método público.
- La flecha gruesa terminada en triángulo indica herencia: `ServicioAPI` y `PlanMensual` son tipos de `OpcionIA`.
- La flecha fina indica una asociación: una clase conoce a la otra.
- `0..*` significa «ninguno o muchos» y `1` significa «exactamente uno».

Así, `Sistema` conoce muchas evaluaciones, y cada `Evaluacion` conoce exactamente una opción y un caso.

3.2 Clase por clase

Clase	Responsabilidad	Atributos	Por qué existe
`Sistema`	Coordina las operaciones y asigna códigos.	Listas de criterios, opciones, casos y evaluaciones.	Da un único punto de acceso a los datos y a las operaciones.
`OpcionIA`	Define lo común a toda opción y exige calcular un costo.	`codigo`, `nombre`, `puntajes`.	Reúne los datos compartidos y permite tratar igual a ambos tipos.
`ServicioAPI`	Calcula el costo según las consultas.	`costo_por_1000`, `consultas_estimadas`.	Su fórmula y sus datos son distintos de los del plan.
`PlanMensual`	Calcula el costo según los usuarios.	`usuarios`, `costo_por_usuario`.	Su fórmula y sus datos son distintos de los del servicio.
`Criterio`	Representa un aspecto a evaluar.	`codigo`, `nombre`, `peso`.	El peso interviene en todos los puntajes.
`CasoUso`	Describe una necesidad con sus límites.	`codigo`, `nombre`, `presupuesto`, `puntaje_minimo`.	Una misma opción puede servir para un caso y no para otro.
`Evaluacion`	Une una opción con un caso y determina el estado.	`codigo`, `opcion`, `caso`, `fecha`, `estado`.	Es el resultado que se lista, se recalcula y se ordena.

Las excepciones propias no aparecen en el diagrama porque, como aclara la letra, no se cuentan como clases del dominio.

> **Decisión: adaptaciones del diagrama**
> El diagrama es el punto de partida, no la lista completa de métodos. La implementación respeta sus clases, atributos y responsabilidades, y agrega los parámetros que cada método necesita:
> - `Evaluacion.puntaje_total(criterios)` recibe la lista de criterios, porque `Evaluacion` no tiene asociación con `Criterio` y necesita conocer los pesos.
> - `Sistema.recalcular(evaluaciones)` recibe las evaluaciones afectadas y le pide a cada una que se recalcule con `Evaluacion.recalcular(criterios)`. Esa clase conoce opción, caso y estado.
> - `Sistema.comparar_caso(caso)` recibe el caso que se compara y, como en el diagrama, no devuelve nada.

3.3 Por qué OpcionIA es abstracta

Una clase abstracta describe algo general que no puede existir por sí solo. Nadie contrata «una opción de IA» sin más: contrata un servicio por consultas o un plan por usuarios. Sin saber el tipo, no hay fórmula para calcular el costo.

Por eso `OpcionIA` hereda de `ABC` y marca `costo_estimado()` con `@abstractmethod`. Python impide crear objetos de una clase abstracta y obliga a cada derivada a implementar ese método. Un **decorador**, escrito con `@` sobre un método, le agrega un comportamiento sin modificar su código.

El polimorfismo es la otra cara de esa decisión: el mismo mensaje produce el cálculo adecuado según el objeto que lo recibe.

entities/OpcionIA.py, líneas 39 a 41

```
    def costo_estimado(self):
        """Costo mensual = (consultas mensuales / 1000) × precio por cada 1000."""
        return self.__consultas_estimadas / 1000 * self.__costo_por_1000
```

entities/OpcionIA.py, líneas 63 a 65

```
    def costo_estimado(self):
        """Costo mensual = usuarios × costo por usuario."""
        return self.__usuarios * self.__costo_por_usuario
```

El texto entre comillas triples es un **docstring**: documenta qué hace el método.

Quien llama escribe siempre `opcion.costo_estimado()`, sin preguntar el tipo. La ventaja es concreta: `Evaluacion` no necesita un `if` para cada clase de opción, y un tercer tipo de opción no obligaría a modificarla.

Las dos derivadas también definen `tipo`, `descripcion_costos()` y `actualizar_costos()`, cada una con sus propios datos. No son abstractos porque la letra solo exige `costo_estimado()`.

3.4 Por qué Evaluacion guarda referencias

La letra indica que las evaluaciones conservan referencias a la opción y al caso. Una referencia no es una copia: `self.__opcion` apunta al mismo objeto que está en la lista de `Sistema`.

Esto tiene una consecuencia práctica. Cuando cambia el precio de una opción, todas las evaluaciones que la referencian ven el nuevo precio sin que nadie las actualice una por una. Con una copia habría dos versiones del dato y una quedaría desactualizada.

3.5 Sistema, el coordinador

`Sistema` no es un concepto del negocio: es quien coordina. Cumple tres funciones:

- Es dueño de las cuatro listas. Nadie más agrega ni elimina elementos.
- Es dueño de los códigos. Mantiene un contador por entidad, como muestra la diapositiva 8.
- Resuelve las reglas que exigen buscar en lo registrado, como la unicidad de nombres.

Un `Criterio` aislado no puede saber si su nombre está repetido, porque no conoce a los demás. `Sistema` sí los conoce, y por eso esa validación es suya.

Una **propiedad** es un método que se lee como un atributo, sin paréntesis (sección 4.5). Las propiedades `criterios`, `opciones` y `casos` permiten que `main.py` lea las listas.

3.6 El diccionario de puntajes

La letra fija el formato: `{codigo_criterio: puntaje}`. Por ejemplo, `{1: 8, 2: 7, 3: 9}` asigna 8 al criterio 1, 7 al criterio 2 y 9 al criterio 3.

La clave es el código y no el nombre porque el código es único e inmutable. Además, el diccionario impide por construcción tener dos puntajes para el mismo criterio. Para calcular el puntaje total se recorre la lista de criterios y se busca cada puntaje por código.

3.7 La interfaz pública de Sistema

Esta tabla es el contrato entre `main.py` y el dominio. Tres métodos aplican una regla y lanzan una excepción cuando se viola: `validar_nombre`, `buscar` y `evaluar`. Lanzar una excepción con `raise` interrumpe la operación. Quien llamó la captura con `try/except`: si el bloque `try` lanza una excepción, la ejecución salta al `except` que corresponde.

Método	Devuelve	Qué hace
`validar_nombre(registros, nombre)`	Nada	Lanza `ErrorLaboratorio` si el nombre ya está.
`buscar(registros, codigo)`	El registro	Lanza `ErrorLaboratorio` si no lo encuentra.
`registrar_criterio(nombre, peso)`	`Criterio`	Registra un criterio.
`registrar_servicio_api(nombre, puntajes, costo_por_1000, consultas)`	`ServicioAPI`	Registra una opción por consultas.
`registrar_plan_mensual(nombre, puntajes, usuarios, costo_por_usuario)`	`PlanMensual`	Registra una opción por usuarios.
`registrar_caso(nombre, presupuesto, puntaje_minimo)`	`CasoUso`	Registra un caso, sin evaluaciones.
`evaluar(opcion, caso)`	`Evaluacion`	Crea una evaluación individual. Lanza `ErrorLaboratorio` si ya existe.
`comparar_caso(caso)`	Nada	Reemplaza las evaluaciones del caso.
`recalcular(evaluaciones)`	Nada	Recalcula el estado de las indicadas.
`actualizar_presupuesto(caso, nuevo)`	Nada	Cambia el presupuesto y recalcula.
`actualizar_costos_opcion(opcion, primer_valor, segundo_valor)`	Nada	Cambia los costos y recalcula.
`evaluaciones_de_caso(caso)`	Una lista	Lista las evaluaciones de un caso.
`ranking_caso(caso)`	Una lista	Ordena las recomendadas.

3.8 Una operación de punta a punta

Seguir una operación ayuda a ver cómo colaboran las clases. Esto ocurre al elegir «Comparar opciones para un caso»:

1. `main.py` lista los casos y lee un código. `Sistema.buscar` devuelve el caso o lanza `ErrorLaboratorio`, y entonces `main.py` vuelve a pedir el código.
2. Si no hay opciones, `main.py` muestra un mensaje y vuelve al menú.
3. `main.py` llama a `sistema.comparar_caso(caso)`.
4. `Sistema` elimina las evaluaciones anteriores de ese caso y conserva las demás.
5. Por cada opción, crea una `Evaluacion` con el siguiente código y la fecha actual.
6. Cada evaluación ejecuta `recalcular`, que pide el costo a la opción y fija el estado.
7. `main.py` muestra las evaluaciones del caso con el formato adecuado.

Cada clase hace solo lo que le corresponde: `main.py` conversa, `Sistema` coordina, `Evaluacion` decide el estado y cada opción calcula su costo.

4 Decisiones de arquitectura

Un buen diseño se puede defender. Este capítulo reúne cada decisión, la alternativa que se descartó y el motivo. Primero se analiza la más importante, porque condiciona a las demás.

4.1 Dominio sin entrada ni salida

Las diapositivas muestran dos versiones de la misma operación.

```
# Diapositiva 9: Sistema recibe los datos ya leídos
def registrar_criterio(self, nombre, peso):
    ...

# Diapositiva 10: Sistema pide los datos por teclado
def registrar_criterio(self):
    nombre = input("Nombre del criterio: ").strip()
    peso = int(input("Peso entero positivo: "))
```

Ambas funcionan. Las diapositivas 11 y 13 también usan `input` dentro de `Sistema`, así que conviene leerlas como pseudocódigo de la lógica, no como ubicación del código de consola.

> **Decisión: se elige la variante de la diapositiva 9**
>
> Las clases de `entities/`, que representan el dominio, no usan `input()` ni `print()`. `main.py` concentra toda la conversación con el usuario. Hay tres motivos:
>
> - Coincide con el diagrama: `Sistema` no tiene responsabilidades de consola.
> - Evita repetir la lectura: `leer_entero` se escribe una vez y la usan todas las operaciones.
> - Separa qué hace el sistema de cómo se conversa con el usuario. Cada mitad se entiende y se corrige por separado.

Lo que aporta la variante descartada es brevedad: el código de cada operación queda junto. El costo aparece después, cuando cada método de `Sistema` debe repetir la lectura de sus datos.

4.2 Tabla de decisiones

Decisión	Alternativa descartada	Por qué
Solo lo que pide la letra: sin validaciones ni features extra.	Agregar pruebas automáticas, topes y controles repetidos.	El código es más fácil de explicar y de corregir, y tiene menos superficie de error.
Dominio sin `input` ni `print`.	`input` dentro de `Sistema` (diapositiva 10).	Respeta el diagrama, evita repetir la lectura y separa responsabilidades.
Cada regla se valida en una sola capa.	Validar lo mismo en la consola y en el dominio.	Una regla escrita dos veces puede quedar distinta. La sección 8.2 indica la capa de cada una.
Una sola excepción propia: `ErrorLaboratorio`.	Varias subclases, o `ValueError` para todo.	La letra pide al menos una, y cada mensaje ya distingue el error. `ValueError` se confundiría con los errores de conversión de `int()`.
Atributos privados con `__` y `@property` de solo lectura.	Atributos públicos, o métodos `get_x()`.	Nadie modifica un dato por accidente desde afuera. Solo se pueden cambiar los que la letra pide actualizar: el presupuesto y los valores de costo.
`Evaluacion` calcula puntaje y costo bajo demanda y solo guarda `estado`.	Guardar puntaje y costo como atributos.	El costo depende de la opción, que puede cambiar: un valor guardado podría quedar viejo. El costo de la alternativa es recalcular en cada lectura, irrelevante a esta escala.
El estado se compara con el puntaje sin redondear.	Redondear a dos decimales antes de comparar.	La letra pide dos decimales solo para mostrar. Redondear antes aceptaría 6.996 con mínimo 7.
Costo contra presupuesto con `round(costo - presupuesto, 6) > 0`.	Comparar con `costo > presupuesto`.	La letra dice que la igualdad es válida, y con precios en centavos hace falta redondear: 5 × 5.99 da 29.950000000000003 frente a 29.95 (sección 9.1).

Decisión	Alternativa descartada	Por qué
Criterios cerrados al existir opciones.	Permitir criterios nuevos y pedir los puntajes que falten.	Es una regla de la letra. Además, un criterio nuevo dejaría opciones sin puntaje y el cálculo fallaría.
Códigos nunca reutilizados, con un contador por entidad.	Calcular el código con `len(lista) + 1`.	Al eliminar evaluaciones la lista se achica y `len` repetiría códigos. La letra lo prohíbe.
Comparar reemplaza todas las evaluaciones del caso.	Agregar las nuevas a las existentes.	Es lo que pide la letra. Acumular duplicaría combinaciones de opción y caso.
Nombre único sin distinguir mayúsculas también en opciones y casos.	Aplicarlo solo a criterios.	La letra lo exige de forma explícita solo para criterios. Un único método resuelve las tres listas, y `Claude` y `CLAUDE` serían casi iguales en un listado.
Nombre validado apenas se lee, como en las diapositivas 11 y 13.	Validar al final, con todos los datos ya ingresados.	El usuario no completa datos en vano.
Los métodos de `Sistema` reciben objetos ya seleccionados.	Recibir códigos y volver a buscarlos.	`main.py` ya obtuvo el objeto al seleccionarlo. Buscarlo otra vez repetiría trabajo.
Registrar opción en el orden de la letra: nombre, tipo, costos y puntajes.	Pedir los puntajes antes de los costos (diapositiva 11).	Ante una diferencia con las diapositivas, se sigue la letra.
Estructura exacta de la letra, sin `__init__.py`.	Agregar archivos `__init__.py` vacíos.	Python importa las carpetas sin ellos, y la letra da la lista exacta de archivos. El riesgo es que otro paquete instalado con el mismo nombre tenga prioridad, algo improbable.
Sin librerías externas: solo `abc` y `datetime`.	Usar paquetes externos.	La biblioteca estándar alcanza, y no hay nada que declarar ni instalar. La letra exige justificar cualquier librería externa.

4.3 La jerarquía de excepciones

La letra exige al menos una excepción propia, y este diseño define una sola: `ErrorLaboratorio`. La lanzan `validar_nombre`, `buscar` y `evaluar`, y cada mensaje indica qué ocurrió. `main.py` la captura en `leer_nombre_nuevo`, `seleccionar_opcion`, `seleccionar_caso` y `registrar_evaluacion`, que son los lugares donde se puede resolver.

4.4 Calcular bajo demanda o guardar

La letra dice que `Evaluacion` registra fecha, puntaje, costo y estado. El diagrama, en cambio, muestra `puntaje_total()` y `costo_total()` como métodos, no como atributos. Este diseño sigue al diagrama: la evaluación ofrece esos valores siempre que se le piden, pero solo guarda `estado`.

Aspecto	Calcular bajo demanda (elegido)	Guardar puntaje y costo
Precio actualizado	Se refleja solo, porque el costo se pide a la opción.	Hay que actualizar cada evaluación, y olvidar una deja datos viejos.
Velocidad	Repite el cálculo en cada lectura.	Lee un valor ya calculado.
Cantidad de código	Menor: un dato vive en un solo lugar.	Mayor: el mismo dato existe en dos lugares.

Al actualizar una opción, la letra pide recalcular el costo y el estado. El costo se recalcula solo, porque se delega en la opción. El estado sí se guarda, y por eso existe el método `recalcular`. El puntaje no corre riesgo de quedar viejo: los criterios se cierran y los puntajes no cambian.

4.5 Encapsulamiento en la práctica

El doble guion bajo, como en la diapositiva 8, vuelve privado el atributo: `self.__peso`. El decorador `@property` ofrece una lectura controlada de ese dato.

Quien usa la clase escribe `criterio.peso`, sin paréntesis. Como `Criterio` no tiene setter, `criterio.peso = 5` produce un error. Cuando la letra exige modificar un dato, como el presupuesto, se agrega un setter:

entities/CasoUso.py, líneas 17 a 23

```
    @property
    def presupuesto(self):
        return self.__presupuesto

    @presupuesto.setter
    def presupuesto(self, nuevo):
        self.__presupuesto = nuevo
```

> **Atención: los atributos con doble guion bajo y la herencia**
> Python renombra internamente `self.__codigo` a `self._OpcionIA__codigo`. Por eso `ServicioAPI` no puede leer `self.__codigo` de su clase padre. Las derivadas deben usar la propiedad: `self.codigo`.

5 Estructura del proyecto y convenciones

5.1 Árbol de carpetas

La letra fija esta estructura (pág. 5). El número de equipo reemplaza a `[numero]`.

```
laboratorio_2026_grupo[numero]/
    main.py
    README.md
    entities/
        Sistema.py
        OpcionIA.py   (incluye ServicioAPI y PlanMensual)
        Criterio.py
        CasoUso.py
        Evaluacion.py
    exceptions/
        ErrorLaboratorio.py
```

5.2 Imports

Un **módulo** es un archivo `.py`, y una carpeta de módulos es un **paquete**. Cada archivo se llama igual que su clase principal, por lo que el nombre se repite en el import. Estos son los imports propios de `main.py`:

main.py, líneas 1 a 3

```
from entities.OpcionIA import ServicioAPI
from entities.Sistema import Sistema
from exceptions.ErrorLaboratorio import ErrorLaboratorio
```

> **Atención**
> `import entities.Sistema` importa el módulo, no la clase. En ese caso, `Sistema()` falla. Hay que usar la forma `from ... import ...`.

5.3 Cómo ejecutar

El programa se ejecuta desde la carpeta `laboratorio_2026_grupo[numero]/`. En sistemas donde `python` no apunta a Python 3, se usa `python3`.

```
python main.py
```

5.4 Mapa de menús

Los menús deben tener los números y textos de la letra (pág. 3). La tabla los relaciona con el método de `Sistema` que usa cada acción.

Menú	Opción	Método de `Sistema`
1. Registrar	1. Criterio	`registrar_criterio`
	2. Opción de IA	`registrar_servicio_api` o `registrar_plan_mensual`
	3. Caso de uso	`registrar_caso`
	4. Evaluación individual	`evaluar`
	5. Salir	Vuelve al menú principal.
2. Comparar opciones para un caso	—	`comparar_caso`
3. Actualizar	1. Presupuesto de un caso	`actualizar_presupuesto`
	2. Precio o consumo de una opción	`actualizar_costos_opcion`
	3. Salir	Vuelve al menú principal.
4. Listar	1. Criterios	Propiedad `criterios`
	2. Opciones	Propiedad `opciones`
	3. Casos de uso	Propiedad `casos` y `ranking_caso`
	4. Evaluaciones de un caso	`evaluaciones_de_caso`
	5. Ranking de un caso	`ranking_caso`
	6. Salir	Vuelve al menú principal.
5. Salir del sistema	—	Finaliza el programa.

5.5 Qué muestra cada listado

La letra define los campos exactos de cada listado (págs. 4 y 5). «Presupuesto actual» es el dato que cambia al actualizar un caso. El mensaje de lista vacía lo pide solo para evaluaciones y ranking; en los demás listados es una decisión de este diseño.

Listado	Campos	Si está vacío
Criterios	Código, nombre y peso.	Mensaje, por decisión propia.
Opciones	Código, nombre, tipo, datos de precio o consumo, costo mensual y puntajes por criterio.	Mensaje, por decisión propia.
Casos de uso	Código, nombre, presupuesto, puntaje mínimo y cantidad de recomendadas actuales.	Mensaje, por decisión propia. Sin evaluaciones, la cantidad es 0.
Evaluaciones de un caso	Código, fecha, opción, presupuesto actual del caso, costo, puntaje total y estado.	Indica que no existen.
Ranking de un caso	Posición, opción, puntaje y costo, solo de las recomendadas.	Indica que no hay.

5.6 Convenciones de código

Elemento	Convención	Ejemplo
Clases	Los nombres de la letra.	`CasoUso`, `OpcionIA`
Métodos y variables	Minúsculas con guion bajo.	`costo_estimado`, `puntaje_minimo`
Atributos	Privados, con doble guion bajo.	`self.__presupuesto`
Constantes	Mayúsculas, de clase o de módulo.	`Evaluacion.RECOMENDADA`, `FORMATO_FECHA`
Idioma	Español neutro en código, comentarios y mensajes.	`"Puntaje insuficiente"`

Se prefieren las construcciones vistas en las diapositivas: `if/elif`, bucles `while`, listas y diccionarios. No hacen falta `match` ni `dataclasses`.

5.7 Formato de números y fechas

Dato	Formato en Python	Se muestra como
Puntaje	`f"{puntaje:.2f}"`	`7.83`
Costo	`f"USD {costo:.2f}"`	`USD 200.00`
Fecha	`fecha.strftime(FORMATO_FECHA)`, con `"%d/%m/%Y %H:%M:%S"`	`02/11/2026 21:30:05`

El formato se aplica solo en `main.py`, al mostrar. Dentro del dominio los números conservan toda su precisión.

6 Plan de construcción en 7 etapas

El plan sigue el orden sugerido en la diapositiva 14. Cada etapa deja un programa que se puede ejecutar: si algo falla, el error está en lo último que se agregó.

6.1 Las siete etapas

Etapa 1: clases del diagrama y excepciones

Objetivo: crear las clases con constructor, propiedades y fórmulas, sin menús.

Archivos: `exceptions/ErrorLaboratorio.py`, `Criterio.py`, `CasoUso.py` y `OpcionIA.py` completos, y en `Evaluacion.py` las constantes, el constructor y las propiedades. Por ahora, `Sistema` solo tiene su constructor.

Esta etapa se comprueba desde el intérprete de Python, abierto en la carpeta del proyecto, porque `main.py` todavía no existe.

Trampas: leer `self.__codigo` del padre desde una derivada; olvidar `@abstractmethod` en `costo_estimado`.

> **Checkpoint**
> Desde el intérprete, sin ejecutar `main.py`, `ServicioAPI(1, "Claude", {1: 8}, 40, 5000).costo_estimado()` devuelve `200.0`, y crear un `OpcionIA` directamente produce `TypeError`.

Etapa 2: menú principal y submenús

Objetivo: navegar todos los menús, aunque las acciones aún no hagan nada.

Archivos: `main.py`. Cada acción puede ser, por ahora, una función con `pass`.

El patrón es el de las diapositivas 2 a 7: un `while` que termina al elegir Salir. La sección 7.9 muestra el menú principal, y los submenús tienen la misma forma.

La lectura de la diapositiva 4 se completa con `try/except`, porque `int("abc")` lanza `ValueError` y terminaría el programa. Como en la diapositiva 7, los menús usan `leer_entero`, que repite la pregunta hasta obtener un número del rango.

Trampas: usar números o textos distintos de los de la letra; crear un `Sistema` nuevo en cada menú en lugar de pasar siempre el mismo.

> **Checkpoint**
> Se puede entrar y salir de cada submenú. Escribir `abc` o `9` muestra un mensaje y vuelve a preguntar.

Etapa 3: operaciones de Registrar

Objetivo: registrar criterios, opciones y casos con sus validaciones.

Archivos: `entities/Sistema.py` (`registrar_*`, `validar_nombre` y las propiedades `criterios`, `opciones` y `casos`) y `main.py` (`leer_flotante`, `leer_texto`, `leer_nombre_nuevo`, `pedir_puntajes` de la diapositiva 12, y las acciones `registrar_*`).

El nombre se valida apenas se lee, como en las diapositivas 11 y 13. La opción se pide en el orden de la letra: nombre, tipo, datos de costo y puntajes. La diapositiva 11 pide los puntajes antes de los costos.

Trampas: comparar nombres sin `lower()`; permitir criterios después de la primera opción; omitir un criterio al pedir los puntajes.

> **Checkpoint**
> `precisión` se rechaza apenas se escribe si ya existe `Precisión`, y se vuelve a pedir el nombre. Tras registrar una opción, un criterio nuevo se rechaza.

Etapa 4: operaciones de Listar

Objetivo: ver lo registrado. Se hace antes de evaluar porque permite comprobar visualmente cada etapa siguiente.

Archivos: `main.py` (`listar_criterios`, `listar_opciones`, `texto_puntajes`, `listar_casos`) y `entities/Sistema.py` (`evaluaciones_de_caso`, `ranking_caso`). `ranking_caso` se escribe aquí porque el listado de casos cuenta sus elementos: devuelve una lista vacía mientras no haya evaluaciones. Los campos están en la sección 5.5.

Trampas: omitir un campo de los que pide la letra; creer que los dos decimales son obligatorios en todo: la letra los exige solo para el puntaje, y aquí se usan en los costos por decisión de este diseño. Los listados de evaluaciones y ranking llegan en las etapas 5 y 6.

> **Checkpoint**
> El listado de opciones muestra código, nombre, tipo, datos de precio o consumo, costo y puntajes. El ejemplo de la letra figura con `USD 200.00`, y cada caso, con 0 recomendadas.

Etapa 5: evaluación individual

Objetivo: crear una evaluación con su estado, mostrarla y listar las evaluaciones de un caso.

Archivos: `entities/Evaluacion.py` (`puntaje_total`, `costo_total`, `recalcular`), `entities/Sistema.py` (`buscar`, `evaluar`, `__crear_evaluacion`) y `main.py` (`seleccionar_opcion`, `seleccionar_caso`, `mostrar_evaluacion`, `mostrar_evaluaciones`, `registrar_evaluacion`, `listar_evaluaciones`).

Trampas: alterar el orden de los estados; redondear antes de comparar; no detectar la combinación duplicada.

> **Checkpoint**
> La evaluación muestra código, fecha, opción, caso, puntaje, costo y estado. Repetir la misma combinación informa el duplicado y no crea otra. El listado de casos ya cuenta las recomendadas.

Etapa 6: comparar opciones

Objetivo: evaluar todas las opciones de un caso de una sola vez y ver su ranking.

Archivos: `entities/Sistema.py` (`comparar_caso`) y `main.py` (`comparar`, `listar_ranking`).

Trampas: eliminar también evaluaciones de otros casos; reiniciar el contador; eliminar elementos de una lista mientras se la recorre. Es más seguro construir una lista nueva con las evaluaciones que se conservan.

> **Checkpoint**
> Si la evaluación individual tenía código 1, al comparar con tres opciones aparecen los códigos 2, 3 y 4. El código 1 no vuelve a usarse.

Etapa 7: actualizar

Objetivo: cambiar presupuestos y costos, y recalcular los estados.

Archivos: `entities/Sistema.py` (`recalcular`, `actualizar_presupuesto`, `actualizar_costos_opcion`) y `main.py` (`actualizar_presupuesto`, `actualizar_precio`).

Trampas: crear evaluaciones nuevas en lugar de recalcular; cambiar la fecha; recalcular solo un caso cuando la opción aparece en varios.

> **Checkpoint**
> Tras subir el presupuesto, el estado cambia y se conservan códigos y fechas. Tras cambiar un precio, el nuevo costo aparece en todos los casos de esa opción.

6.2 Qué reglas cubre cada etapa

Esta tabla relaciona el plan con las reglas del capítulo 2. Sirve para comprobar que ninguna quedó sin atender. La regla R21, que pide reintentar ante un error, se aplica en todas las etapas que leen datos.

Etapa	Resultado	Reglas
1	Clases, fórmulas de costo y excepciones.	R7
2	Navegación completa por los menús.	R22
3	Registro de criterios, opciones y casos.	R1 a R6, R8, R9
4	Listados de criterios, opciones y casos.	R19
5	Evaluación individual y evaluaciones de un caso.	R10 a R12
6	Comparación y ranking.	R13 a R15, R20
7	Actualización de presupuestos y costos.	R16 a R18

Al final de cada etapa hay un punto de integración: se unen los cambios y quien no los programó ejecuta el checkpoint. Conviene acordar primero los nombres de los métodos de `Sistema`, porque son el contrato entre el dominio y la consola.

7 Implementación comentada

Este capítulo recorre la implementación de referencia, desde las clases más simples hasta la consola. Cada bloque se copió de forma automática del código real. La línea sobre cada bloque indica de qué archivo y de qué líneas proviene.

7.1 Excepciones propias

La letra exige al menos una excepción propia (pág. 2). Alcanza con una clase, como se explica en la sección 4.3:

exceptions/ErrorLaboratorio.py, líneas 1 a 2

```
class ErrorLaboratorio(Exception):
    pass
```

- `pass` indica que la clase no agrega nada: heredar de `Exception` aporta todo lo necesario.
- El mensaje se escribe al lanzar la excepción, donde se conoce el dato concreto. `main.py` solo lo muestra con `print(f"Error: {error}")`.

7.2 Criterio y CasoUso

`Criterio` guarda tres datos y permite leerlos. Es la clase más simple del dominio.

entities/Criterio.py, líneas 1 a 18

```
class Criterio:

    def __init__(self, codigo, nombre, peso):
        self.__codigo = codigo
        self.__nombre = nombre
        self.__peso = peso

    @property
    def codigo(self):
        return self.__codigo

    @property
    def nombre(self):
        return self.__nombre

    @property
    def peso(self):
        return self.__peso
```

- El constructor guarda los datos sin validarlos: `main.py` ya controló el peso con `leer_entero` y el nombre con `leer_texto` (sección 8.2).
- Cada atributo privado se lee con su propiedad. Como no hay setter, nadie puede cambiarlos desde afuera.

`CasoUso` tiene la misma forma, con `codigo`, `nombre`, `presupuesto` y `puntaje_minimo`. Solo `presupuesto` tiene setter, que se mostró en la sección 4.5, porque la letra no prevé modificar los demás datos.

7.3 OpcionIA, ServicioAPI y PlanMensual (abstracción y polimorfismo)

La clase abstracta reúne los datos comunes y exige un solo método a sus derivadas (R7):

entities/OpcionIA.py, líneas 4 a 25

```python
class OpcionIA(ABC):

    def __init__(self, codigo, nombre, puntajes):
        self.__codigo = codigo
        self.__nombre = nombre
        self.__puntajes = puntajes

    @property
    def codigo(self):
        return self.__codigo

    @property
    def nombre(self):
        return self.__nombre

    @property
    def puntajes(self):
        return self.__puntajes

    @abstractmethod
    def costo_estimado(self):
        pass
```

- Hereda de `ABC`, y por eso Python rechaza `OpcionIA(...)` con `TypeError`.
- `costo_estimado` no tiene código en `OpcionIA`: cada derivada debe escribir el suyo.

Las derivadas agregan sus datos propios. `PlanMensual` tiene la misma forma, con `usuarios` y `costo_por_usuario`.

entities/OpcionIA.py, líneas 28 a 49

```python
class ServicioAPI(OpcionIA):

    def __init__(self, codigo, nombre, puntajes, costo_por_1000, consultas_estimadas):
        super().__init__(codigo, nombre, puntajes)
        self.__costo_por_1000 = costo_por_1000
        self.__consultas_estimadas = consultas_estimadas

    @property
    def tipo(self):
        return "Servicio API"

    def costo_estimado(self):
        """Costo mensual = (consultas mensuales / 1000) × precio por cada 1000."""
        return self.__consultas_estimadas / 1000 * self.__costo_por_1000

    def descripcion_costos(self):
        return (f"USD {self.__costo_por_1000:.2f} por cada 1000 consultas, "
                f"{self.__consultas_estimadas} consultas estimadas")

    def actualizar_costos(self, costo_por_1000, consultas_estimadas):
        self.__costo_por_1000 = costo_por_1000
        self.__consultas_estimadas = consultas_estimadas
```

- `super().__init__(...)` delega la parte común en `OpcionIA`.
- `actualizar_costos` recibe los dos valores propios de cada tipo, en el orden de la letra. `Sistema` los pasa como `primer_valor` y `segundo_valor`, y `actualizar_precio`, en `main.py`, usa `isinstance(opcion, ServicioAPI)` para saber qué preguntar. El dominio no pregunta el tipo en ningún caso.

7.4 Evaluacion y el cálculo del estado

Los tres estados son constantes de clase. Un error de escritura en una constante produce un `AttributeError` visible, mientras que un texto mal escrito pasaría inadvertido.

entities/Evaluacion.py, líneas 3 a 5

```
    RECOMENDADA = "Recomendada"
    FUERA_DE_PRESUPUESTO = "Fuera de presupuesto"
    PUNTAJE_INSUFICIENTE = "Puntaje insuficiente"
```

Puntaje y costo se calculan cada vez que se piden (sección 4.4):

entities/Evaluacion.py, líneas 34 a 48

```
    def puntaje_total(self, criterios):
        """Devuelve suma(puntaje × peso) / suma de pesos, sin redondear."""
        puntajes = self.__opcion.puntajes
        suma_ponderada = 0
        suma_pesos = 0
        posicion = 0
        while posicion < len(criterios):
            criterio = criterios[posicion]
            suma_ponderada += puntajes[criterio.codigo] * criterio.peso
            suma_pesos += criterio.peso
            posicion += 1
        return suma_ponderada / suma_pesos

    def costo_total(self):
        return self.__opcion.costo_estimado()
```

- `puntaje_total` aplica la fórmula de la pág. 4. Recorre los criterios y busca cada puntaje por código.
- `costo_total` delega en la opción, y el polimorfismo elige la fórmula.

El estado sí se guarda, y `recalcular` es el único método que lo modifica:

entities/Evaluacion.py, líneas 50 a 58

```
    def recalcular(self, criterios):
        """Fija el estado en el orden de la letra; la igualdad con cada límite es válida."""
        # Redondeo para que 5 × 5.99 sea igual a 29.95.
        if round(self.costo_total() - self.__caso.presupuesto, 6) > 0:
            self.__estado = Evaluacion.FUERA_DE_PRESUPUESTO
        elif self.puntaje_total(criterios) < self.__caso.puntaje_minimo:
            self.__estado = Evaluacion.PUNTAJE_INSUFICIENTE
        else:
            self.__estado = Evaluacion.RECOMENDADA
```

- El orden de `if`, `elif` y `else` reproduce el orden de la letra.
- `round(costo - presupuesto, 6) > 0` hace que un costo igual al presupuesto sea válido también con precios en centavos (sección 9.1).
- `<` es estricto, de modo que un puntaje igual al mínimo es válido. La comparación usa el puntaje completo, sin redondear.

7.5 Sistema: registrar y validar

`validar_nombre` controla la unicidad en un solo método, que sirve para las tres listas. Compara con `lower()`, porque `leer_texto` ya quitó los espacios de los extremos.

entities/Sistema.py, líneas 34 a 40

```
    def validar_nombre(self, registros, nombre):
        """Lanza ErrorLaboratorio si el nombre ya está en la lista, sin distinguir mayúsculas."""
        posicion = 0
        while posicion < len(registros):
            if registros[posicion].nombre.lower() == nombre.lower():
                raise ErrorLaboratorio(f"Ya existe un registro llamado {nombre}.")
            posicion += 1
```

La letra exige la insensibilidad a mayúsculas solo para criterios, y este diseño la extiende a opciones y casos (sección 4.2). `main.py` llama a este método apenas lee el nombre, antes de pedir los demás datos.

Los registros siguen un mismo esquema, como en la diapositiva 9: crear el objeto con el próximo código, agregarlo a la lista y avanzar el contador.

entities/Sistema.py, líneas 51 a 55

```
    def registrar_criterio(self, nombre, peso):
        criterio = Criterio(self.__proximo_codigo_criterio, nombre, peso)
        self.__criterios.append(criterio)
        self.__proximo_codigo_criterio += 1
        return criterio
```

Las opciones repiten el esquema. `registrar_plan_mensual` es igual, salvo por la clase que crea.

entities/Sistema.py, líneas 57 a 62

```
    def registrar_servicio_api(self, nombre, puntajes, costo_por_1000, consultas):
        opcion = ServicioAPI(self.__proximo_codigo_opcion, nombre, puntajes,
                             costo_por_1000, consultas)
        self.__opciones.append(opcion)
        self.__proximo_codigo_opcion += 1
        return opcion
```

7.6 Sistema: evaluar y comparar

La evaluación individual controla el duplicado antes de crear nada (R10):

entities/Sistema.py, líneas 77 a 87

```
    def evaluar(self, opcion, caso):
        """Crea una evaluación individual; rechaza la combinación ya evaluada."""
        existentes = self.evaluaciones_de_caso(caso)
        posicion = 0
        while posicion < len(existentes):
            if existentes[posicion].opcion == opcion:
                raise ErrorLaboratorio(
                    f"Ya existe una evaluación de {opcion.nombre} para {caso.nombre}."
                )
            posicion += 1
        return self.__crear_evaluacion(opcion, caso)
```

La comparación reemplaza las evaluaciones del caso (R13 a R15):

entities/Sistema.py, líneas 89 a 103

```
    def comparar_caso(self, caso):
        """Reemplaza todas las evaluaciones del caso por una nueva por cada opción."""
        conservadas = []
        posicion = 0
        while posicion < len(self.__evaluaciones):
            evaluacion = self.__evaluaciones[posicion]
            if evaluacion.caso != caso:
                conservadas.append(evaluacion)
            posicion += 1
        self.__evaluaciones = conservadas

        posicion = 0
        while posicion < len(self.__opciones):
            self.__crear_evaluacion(self.__opciones[posicion], caso)
            posicion += 1
```

- En lugar de eliminar elementos mientras se recorre la lista, construye `conservadas` con lo que debe quedar. Es más simple y evita omitir posiciones.
- Se asigna una lista nueva a `self.__evaluaciones`, pero el contador no retrocede. Por eso los códigos no se reutilizan.

Las dos operaciones crean evaluaciones con el mismo método. Así se cumple que ambas usen las mismas reglas de cálculo (pág. 4):

entities/Sistema.py, líneas 154 a 159

```
    def __crear_evaluacion(self, opcion, caso):
        evaluacion = Evaluacion(self.__proximo_codigo_evaluacion, opcion, caso, datetime.now())
        evaluacion.recalcular(self.__criterios)
        self.__evaluaciones.append(evaluacion)
        self.__proximo_codigo_evaluacion += 1
        return evaluacion
```

7.7 Sistema: actualizar y recalcular

`recalcular` es la adaptación del método público del diagrama (sección 3.2). Recibe las evaluaciones afectadas, que cada actualización calcula a su manera.

entities/Sistema.py, líneas 105 a 126

```
    def recalcular(self, evaluaciones):
        posicion = 0
        while posicion < len(evaluaciones):
            evaluaciones[posicion].recalcular(self.__criterios)
            posicion += 1

    def actualizar_presupuesto(self, caso, nuevo):
        caso.presupuesto = nuevo
        self.recalcular(self.evaluaciones_de_caso(caso))

    def actualizar_costos_opcion(self, opcion, primer_valor, segundo_valor):
        """Cambia los costos y recalcula las evaluaciones de la opción en todos los casos."""
        opcion.actualizar_costos(primer_valor, segundo_valor)

        afectadas = []
        posicion = 0
        while posicion < len(self.__evaluaciones):
            evaluacion = self.__evaluaciones[posicion]
            if evaluacion.opcion == opcion:
                afectadas.append(evaluacion)
            posicion += 1
        self.recalcular(afectadas)
```

- El presupuesto afecta a un solo caso. El costo afecta a la opción en todos los casos, y por eso se recorre la lista completa (R16 y R18).
- `Evaluacion.recalcular` solo escribe el estado, así que códigos y fechas no se tocan.

7.8 Sistema: ranking

El ranking implementa R20: primero filtra las recomendadas y después las ordena con una clave.

entities/Sistema.py, líneas 138 a 152

```
    def ranking_caso(self, caso):
        """Ordena las recomendadas por mayor puntaje, menor costo y menor código de opción."""
        evaluaciones = self.evaluaciones_de_caso(caso)
        recomendadas = []
        posicion = 0
        while posicion < len(evaluaciones):
            if evaluaciones[posicion].estado == Evaluacion.RECOMENDADA:
                recomendadas.append(evaluaciones[posicion])
            posicion += 1

        criterios = self.__criterios
        return sorted(
            recomendadas,
            key=lambda e: (-e.puntaje_total(criterios), round(e.costo_total(), 6), e.opcion.codigo),
        )
```

- `lambda` define una función breve sin nombre: recibe una evaluación y devuelve su clave.
- Python compara las tuplas elemento por elemento, y pasa al siguiente solo si hay empate. Esa regla coincide con los desempates de la letra.
- El signo negativo invierte el orden del puntaje: el mayor puntaje produce el menor valor y queda primero.
- El costo se redondea a seis decimales para que el ruido de los `float` no rompa un empate (sección 9.3).

- `sorted` devuelve una lista nueva, sin alterar la original.
- El listado de casos usa `len(sistema.ranking_caso(caso))` para mostrar la cantidad de recomendadas.

7.9 main.py: lectura robusta y menús

`main.py` concentra la consola y se apoya en funciones auxiliares: `leer_*` repiten hasta obtener un dato válido, `seleccionar_opcion` y `seleccionar_caso` listan y piden un código, `pedir_puntajes` pide un puntaje por criterio, y `menu_registrar`, `menu_actualizar` y `menu_listar` son los submenús.

Las funciones de lectura extienden el patrón de la diapositiva 4 (R21). `leer_flotante` tiene la misma forma que `leer_entero`, con `float` en lugar de `int` y con el mínimo obligatorio.

main.py, líneas 8 a 23

```
def leer_entero(mensaje, minimo=None, maximo=None):
    """Repite la lectura hasta obtener un entero dentro del rango indicado."""
    valido = False
    valor = 0
    while not valido:
        try:
            valor = int(input(mensaje))
            if minimo is not None and valor < minimo:
                print(f"Ingrese un número mayor o igual que {minimo}.")
            elif maximo is not None and valor > maximo:
                print(f"Ingrese un número menor o igual que {maximo}.")
            else:
                valido = True
        except ValueError:
            print("Ingrese un número entero.")
    return valor
```

Los parámetros con valor `None` son opcionales: `leer_entero("Peso entero positivo: ", 1)` pide un mínimo y ningún máximo.

`leer_nombre_nuevo` rechaza un nombre repetido apenas se escribe, como en las diapositivas 11 y 13. Recibe la lista en la que debe buscar:

main.py, líneas 53 a 64

```
def leer_nombre_nuevo(sistema, mensaje, registros):
    """Repite la lectura hasta obtener un nombre que no esté en la lista."""
    valido = False
    nombre = ""
    while not valido:
        nombre = leer_texto(mensaje)
        try:
            sistema.validar_nombre(registros, nombre)
            valido = True
        except ErrorLaboratorio as error:
            print(f"Error: {error}")
    return nombre
```

`seleccionar_caso` combina un listado con una búsqueda. Devuelve `None` si no hay casos, y cada acción termina en ese caso.

main.py, líneas 82 a 94

```
def seleccionar_caso(sistema):
    listar_casos(sistema)
    if len(sistema.casos) == 0:
        return None

    caso = None
    while caso is None:
        codigo = leer_entero("Código del caso de uso: ")
        try:
            caso = sistema.buscar(sistema.casos, codigo)
        except ErrorLaboratorio as error:
            print(f"Error: {error}")
    return caso
```

Cada acción del menú combina lectura y dominio. `registrar_opcion` muestra el patrón completo:

main.py, líneas 160 a 177

```
def registrar_opcion(sistema):
    if len(sistema.criterios) == 0:
        print("Debe existir al menos un criterio para registrar opciones.")
        return

    nombre = leer_nombre_nuevo(sistema, "Nombre de la opción: ", sistema.opciones)
    tipo = leer_entero("Tipo (1. Servicio API, 2. Plan mensual): ", 1, 2)
    if tipo == 1:
        precio = leer_flotante("Precio en USD por cada 1000 consultas: ", 0)
        consultas = leer_entero("Consultas mensuales estimadas: ", 1)
        puntajes = pedir_puntajes(sistema.criterios)
        opcion = sistema.registrar_servicio_api(nombre, puntajes, precio, consultas)
    else:
        usuarios = leer_entero("Cantidad de usuarios: ", 1)
        costo = leer_flotante("Costo mensual en USD por usuario: ", 0)
        puntajes = pedir_puntajes(sistema.criterios)
        opcion = sistema.registrar_plan_mensual(nombre, puntajes, usuarios, costo)
    print(f"Opción registrada con código {opcion.codigo}.")
```

- Pide nombre, tipo, datos de costo y puntajes, como la letra. La diapositiva 11 pide los puntajes antes.
- Cuando llama a `Sistema`, todos los datos ya están validados. Por eso no necesita `try`.

Los listados siguen un mismo esquema: eligen el dato, informan si no hay nada y recorren la lista. `listar_ranking` muestra los campos de la sección 5.5:

main.py, líneas 294 a 311

```
def listar_ranking(sistema):
    caso = seleccionar_caso(sistema)
    if caso is None:
        return

    ranking = sistema.ranking_caso(caso)
    if len(ranking) == 0:
        print(f"No hay evaluaciones recomendadas para {caso.nombre}.")
        return

    print(f"\nRanking de {caso.nombre}:")
    posicion = 0
    while posicion < len(ranking):
        evaluacion = ranking[posicion]
        print(f"{posicion + 1}. {evaluacion.opcion.nombre}"
              f" | Puntaje: {evaluacion.puntaje_total(sistema.criterios):.2f}"
              f" | Costo: USD {evaluacion.costo_total():.2f}")
        posicion += 1
```

`posicion + 1` hace que la posición empiece en 1. El formato `:.2f` se aplica aquí, solo para mostrar.

El menú principal usa el mismo `while` que los submenús. Crea un único `Sistema` y lo pasa como parámetro a cada acción:

main.py, líneas 377 a 403

```
def main():
    sistema = Sistema()
    opcion = 0
    while opcion != 5:
        print("\nMENÚ PRINCIPAL")
        print("1. Registrar")
        print("2. Comparar opciones para un caso")
        print("3. Actualizar")
        print("4. Listar")
        print("5. Salir del sistema")

        opcion = leer_entero("Seleccione una opción: ", 1, 5)

        if opcion == 1:
            menu_registrar(sistema)
        elif opcion == 2:
            comparar(sistema)
        elif opcion == 3:
            menu_actualizar(sistema)
        elif opcion == 4:
            menu_listar(sistema)

    print("Programa finalizado.")


if __name__ == "__main__":
    main()
```

Las dos últimas líneas arrancan el programa, como en la diapositiva 7: sin ellas, `python main.py` define las funciones y termina sin hacer nada. `__name__` vale `"__main__"` solo cuando el archivo se ejecuta directamente, no cuando otro archivo lo importa.

8 Validaciones y manejo de errores

La letra pide validar tipos de datos, campos vacíos, rangos, códigos inexistentes y nombres duplicados, y solicitar de nuevo el dato sin finalizar el programa. El diseño asigna cada regla a una sola capa, de modo que ninguna se escribe dos veces.

8.1 Una capa por regla

`main.py` valida lo que llega por teclado. Funciones como `leer_entero` repiten la pregunta hasta obtener un dato del tipo y el rango esperados. El error de tipo se resuelve aquí, capturando `ValueError`. `main.py` también comprueba, antes de pedir datos, lo que se ve con solo mirar el largo de una lista: si los criterios están cerrados, si hay criterios y si hay opciones para comparar.

`Sistema` valida lo que exige buscar en lo registrado. Es el caso del nombre duplicado, el código inexistente y la evaluación repetida. Cuando se viola una regla, lanza `ErrorLaboratorio`, y `main.py` la captura y muestra su mensaje. Lo que ocurre después depende de la regla:

- Ante un nombre duplicado, `leer_nombre_nuevo` vuelve a pedir el nombre.
- Ante un código inexistente, `seleccionar_opcion` y `seleccionar_caso` vuelven a pedir el código.
- Ante una evaluación repetida, se muestra el mensaje y se vuelve al submenú.

Un ejemplo con un nombre repetido muestra el recorrido completo:

1. `leer_nombre_nuevo` obtiene `precisión` con `leer_texto`.
2. Llama a `sistema.validar_nombre(sistema.criterios, "precisión")`, que compara con `lower()` y lanza `ErrorLaboratorio`.
3. `main.py` captura la excepción, muestra su mensaje y vuelve a pedir el nombre, sin haber pedido el peso.
4. Con un nombre libre, `main.py` lee el peso y llama a `registrar_criterio`.

El programa nunca se detuvo, y `Sistema` no imprimió nada.

Decisión: cada regla se valida una sola vez

Las entidades no repiten los controles de `main.py`. Reciben datos ya validados, porque `main.py` es el único que las usa. Validar dos veces obligaría a mantener dos versiones de la misma regla, con mensajes distintos, sin que el programa se comporte de otra manera.

8.2 Catálogo de validaciones

Dato	Regla	Capa que la valida	Si no se cumple
Opción de menú o tipo	Entero dentro del rango indicado.	`main.py`: `leer_entero`	Reintento con mensaje.
Nombre	No vacío.	`main.py`: `leer_texto`	Reintento con mensaje.
Nombre	Único en su lista, sin distinguir mayúsculas.	`Sistema`: `validar_nombre`	`ErrorLaboratorio`; reintento.
Peso, consultas, usuarios	Entero mayor que cero.	`main.py`: `leer_entero`	Reintento con mensaje.
Puntajes	Un entero de 0 a 10 por cada criterio.	`main.py`: `pedir_puntajes`	Reintento con mensaje.
Costo unitario, presupuesto	Número mayor o igual que cero.	`main.py`: `leer_flotante`	Reintento con mensaje.
Puntaje mínimo	Número entre 0 y 10.	`main.py`: `leer_flotante`	Reintento con mensaje.
Código	Debe estar registrado.	`Sistema`: `buscar`	`ErrorLaboratorio`; reintento.
Evaluación	Sin repetir la combinación de opción y caso.	`Sistema`: `evaluar`	`ErrorLaboratorio`; vuelta al submenú.
Nuevo criterio	No se admite si ya hay opciones.	`main.py`: `registrar_criterio`	Mensaje y vuelta al submenú.
Nueva opción	Debe existir al menos un criterio.	`main.py`: `registrar_opcion`	Mensaje y vuelta al submenú.
Comparar	Debe haber opciones.	`main.py`: `comparar`	Mensaje y vuelta al menú.
Selección	Debe haber registros para elegir.	`main.py`: `seleccionar_*`	Mensaje y vuelta al submenú.

Atención: un except demasiado amplio esconde errores

Capturar una excepción propia es seguro porque solo representa errores previstos. Un `except Exception` también ocultaría errores de programación, como un atributo mal escrito, y dejaría el programa funcionando de forma incorrecta.

9 Casos borde y errores frecuentes

Los casos borde están en los límites de cada regla, y allí suelen aparecer los errores que no se ven al probar con datos normales. La siguiente tabla reúne los más frecuentes.

Situación	Comportamiento correcto	Error típico
Costo igual al presupuesto	Es válido. Un costo de USD 200 con presupuesto de USD 200 no queda «Fuera de presupuesto».	Comparar con `>=`, o comparar sin redondear: con precios en centavos, la igualdad se pierde.
Puntaje igual al mínimo	Es válido. Un puntaje de 7 con mínimo 7 no queda como «Puntaje insuficiente».	Usar `<=` en lugar de `<` al comparar el puntaje.
Puntaje 6.996 con mínimo 7	«Puntaje insuficiente», aunque se muestre como `7.00`.	Redondear antes de comparar.
Orden de los estados	Primero el presupuesto. Una opción cara y con poco puntaje es «Fuera de presupuesto».	Evaluar el puntaje en primer lugar.
Comparar sin opciones	`main.py` muestra un mensaje y vuelve al menú.	Dejar al usuario sin ninguna respuesta.
Caso sin evaluaciones	El listado muestra cantidad 0 de recomendadas. Evaluaciones de un caso indica que no existen.	Mostrar una tabla vacía, o fallar por una búsqueda sin resultado.
Ranking sin recomendadas	Se informa con un mensaje. Las demás evaluaciones no aparecen.	Ordenar todas las evaluaciones del caso sin filtrar.
Actualizar un precio	Se recalculan las evaluaciones de esa opción en todos los casos. Los puntajes no cambian.	Recalcular solo un caso.
Actualizar un presupuesto	Solo cambian el presupuesto y los estados del caso. No se crean evaluaciones ni cambian fechas. Sin evaluaciones, solo se actualiza el presupuesto.	Llamar a la comparación, que reemplaza evaluaciones y fechas.
Nombre duplicado	`Precisión`, `PRECISIÓN` y `precisión` son el mismo nombre. La letra lo exige para criterios, y este diseño lo extiende a opciones y casos.	Comparar los textos tal como se ingresaron.
Códigos tras eliminar	Si la última evaluación fue la 4, la siguiente es la 5, aunque la lista tenga menos elementos.	Usar `len(lista) + 1` o reiniciar el contador.
Peso cero	Se rechaza. Como cada peso es positivo y hay al menos un criterio, la suma de pesos nunca es cero.	Aceptarlo y arriesgar un `ZeroDivisionError`.
Puntajes incompletos	No pueden ocurrir: `pedir_puntajes` recorre todos los criterios y pide un puntaje por cada uno.	Armar el diccionario sin recorrer la lista de criterios.

9.1 El costo igual al presupuesto

La letra dice que un costo igual al presupuesto es válido. Con precios en centavos, comparar con `>` no alcanza: Python guarda los `float` en binario, y 5 × 5.99 da 29.950000000000003 frente a 29.95. Una opción de 5 usuarios a USD 5.99 quedaría «Fuera de presupuesto» con un presupuesto de USD 29.95, aunque la pantalla muestre USD 29.95 en ambos lados. Por eso `recalcular` compara `round(costo - presupuesto, 6) > 0`: el redondeo a seis decimales es necesario para cumplir la letra, y un centavo de más sigue quedando fuera.

9.2 Un ejemplo de 6.996

El caso 6.996 no es teórico. Puede producirse con dos criterios de pesos 249 y 1, y puntajes 7 y 6:

$$\frac{7 \times 249 + 6 \times 1}{249 + 1} = \frac{1749}{250} = 6.996$$

El formato `f"{puntaje:.2f}"` muestra `7.00`, pero el valor real no alcanza el mínimo. Por eso la pantalla mostrará `7.00` junto a «Puntaje insuficiente». No es un error: el dato mostrado está redondeado y la decisión se toma con el valor completo.

9.3 Empates en el ranking

El ranking usa el puntaje sin redondear. Dos opciones que se muestran con `7.83` pueden no estar empatadas si sus puntajes reales difieren. En ese caso, la de mayor puntaje real va primero, sin mirar el costo.

El costo, en cambio, se redondea a seis decimales, por la misma razón que en la sección 9.1. Un plan de 3 usuarios a USD 1.10 cuesta 3.3000000000000003, y uno de 1 usuario a USD 3.30 cuesta 3.3. Ambos cuestan USD 3.30, así que empatan y se resuelven por código de opción.

Cuando el puntaje coincide, se aplican los desempates en orden. Este ejemplo muestra los tres criterios en acción:

Posición	Motivo	Puntaje	Costo	Código de opción
1	Tiene el mayor puntaje.	9.00	USD 300.00	4
2	Empata en puntaje, pero tiene el menor costo.	8.00	USD 50.00	2
3	Empata en puntaje y costo, pero tiene el menor código.	8.00	USD 100.00	1
4	Empata en puntaje y costo con la anterior.	8.00	USD 100.00	3

9.4 Limitaciones conocidas

Hay una limitación aceptada a propósito para no agregar código. La letra pide validar rangos y no terminar el programa, y con `nan`, `inf` o cantidades de cientos de dígitos el programa no lo cumple: `nan` e `inf` se aceptan como números, y una cantidad así lo termina con un error de Python al calcular el costo. Un corrector difícilmente lo pruebe. EOF o Ctrl+C durante una lectura también terminan el programa.

10 Cómo probar

Probar no es ejecutar una vez y ver que no se rompe. Es comprobar, con datos elegidos a propósito, que cada regla se cumple. La letra no pide pruebas automáticas, así que la prueba del programa es este guion por consola.

10.1 Guion de prueba manual

El guion tiene dos partes, y cada una empieza con el programa recién iniciado.

Parte A: recorrido completo

Esta parte usa el ejemplo de la letra y dos casos de uso. Los datos están elegidos para que aparezcan los tres estados, y los pasos de listado permiten ver cada resultado. Los puntajes se ingresan en el orden Precisión, Velocidad y Soporte, con pesos 3, 2 y 1.

Paso	Acción	Entrada	Resultado esperado
1	Menú principal	`abc`, luego `9`	Mensaje de error en ambos casos, sin terminar el programa.
2	Registrar criterio	`Precisión`, 3	Código 1.
3	Registrar criterio	`Velocidad`, 2	Código 2.
4	Registrar criterio	`precisión`; luego `Soporte`, 1	Rechaza el nombre sin pedir el peso. Después, código 3.
5	Listar criterios	—	`[1] Precisión (peso 3)`, `[2] Velocidad (peso 2)` y `[3] Soporte (peso 1)`.
6	Registrar opción	`Claude`, tipo 1, 40, 5000, puntajes 8, 7, 9	Código 1.
7	Registrar opción	`OpenAI`, tipo 2, 10 usuarios, 15, puntajes 6, 7, 5	Código 2.
8	Registrar opción	`Gemini`, tipo 1, 30, 10000, puntajes 9, 8, 8	Código 3.
9	Listar opciones	—	Costos de USD 200.00, 150.00 y 300.00, con los puntajes de cada criterio.
10	Registrar criterio	—	Avisa que ya hay opciones registradas. No pide datos.
11	Registrar caso	`Atención al cliente`, 250, 7	Código 1.
12	Registrar caso	`Análisis de documentos`, 300, 7.5	Código 2.
13	Listar casos	—	`Recomendadas: 0` en ambos. Registrar un caso no crea evaluaciones.
14	Evaluación individual	Opción 1, caso 1	Código 1, puntaje 7.83, USD 200.00, Recomendada.
15	Evaluación individual	Opción 1, caso 1	Informa el duplicado. No crea otra.
16	Comparar	Caso 1	Códigos 2, 3 y 4. Claude: 7.83, Recomendada. OpenAI: 6.17, Puntaje insuficiente. Gemini: 8.50, Fuera de presupuesto.
17	Listar casos	—	Caso 1: `Recomendadas: 1`. Caso 2: `Recomendadas: 0`.
18	Comparar	Caso 2	Códigos 5, 6 y 7. Claude: Recomendada. OpenAI: Puntaje insuficiente. Gemini: Recomendada.
19	Evaluaciones de un caso	Caso 1	Siguen los códigos 2, 3 y 4: comparar el caso 2 las conservó. `Presupuesto: USD 250.00`. Anotar las fechas.
20	Actualizar presupuesto	Caso 1, 300	`Presupuesto actualizado.`
21	Evaluaciones de un caso	Caso 1	`Presupuesto: USD 300.00`. Gemini pasa a Recomendada: su costo es igual al presupuesto. Códigos y fechas no cambian.

Paso	Acción	Entrada	Resultado esperado
22	Ranking	Caso 1	1. Gemini, 8.50, USD 300.00. 2. Claude, 7.83, USD 200.00.
23	Actualizar opción	Opción 1, 70, 5000	`Opción actualizada.`
24	Listar opciones	—	Claude cuesta USD 350.00.
25	Evaluaciones de un caso	Caso 1	Claude: USD 350.00, puntaje 7.83, Fuera de presupuesto. Códigos 2, 3 y 4.
26	Evaluaciones de un caso	Caso 2	Claude: USD 350.00, puntaje 7.83, Fuera de presupuesto. Códigos 5, 6 y 7: actualizar una opción recalcula en todos los casos.
27	Ranking	Caso 1	Solo Gemini.
28	Salir del sistema	5	El programa finaliza sin errores.

Parte B: casos borde

Esta parte reúne los bordes de los capítulos 2 y 9. Usa otros criterios porque los pesos 249 y 1 son los que permiten obtener 6.996. Los puntajes se ingresan en el orden Exactitud y Rapidez.

Paso	Acción	Entrada	Resultado esperado
1	Registrar criterio	`Exactitud`, 249	Código 1.
2	Registrar criterio	`Rapidez`, 1	Código 2.
3	Registrar opción	`Justa`, tipo 2, 5 usuarios, 5.99, puntajes 7, 7	Código 1.
4	Registrar opción	`Casi`, tipo 2, 1 usuario, 10, puntajes 7, 6	Código 2.
5	Registrar opción	`Barata`, tipo 2, 1 usuario, 20, puntajes 7, 7	Código 3.
6	Registrar caso	`Bordes`, 29.95, 7	Código 1.
7	Comparar	Caso 1	`Justa`: `Costo: USD 29.95`, `Puntaje: 7.00`, `Estado: Recomendada`. Su costo es igual al presupuesto y su puntaje, igual al mínimo. `Casi`: `Costo: USD 10.00`, `Puntaje: 7.00`, `Estado: Puntaje insuficiente`. Su puntaje real es 6.996. `Barata`: `Costo: USD 20.00`, `Puntaje: 7.00`, `Estado: Recomendada`.

8	Ranking	Caso 1	`1. Barata | Puntaje: 7.00 | Costo: USD 20.00`
			`2. Justa | Puntaje: 7.00 | Costo: USD 29.95`
			El empate de puntaje se resuelve por menor costo.

9	Salir del sistema	5	El programa finaliza sin errores.

11 Entrega y checklist final

11.1 README

La letra exige instrucciones de ejecución y el número de equipo. Alcanza con esto:

- `Nro de equipo: ____`.
- Requisitos: Python `>= 3.10`, sin librerías externas.
- Cómo ejecutar el programa.
- Una línea con la estructura.

11.2 Archivo comprimido

La letra fija el nombre de la carpeta, `laboratorio_2026_grupo[numero]/`, y pide entregar un archivo comprimido. Lo más coherente es que el archivo lleve el mismo nombre, por ejemplo `laboratorio_2026_grupo7.zip` para el equipo 7.

El comprimido incluye toda la carpeta y se envía por mail. Antes de enviarlo, conviene descomprimirlo en otra carpeta y ejecutar `python main.py`. Es la mejor forma de detectar un archivo faltante. Conviene excluir las carpetas `__pycache__`.

11.3 Checklist final

Funcionalidad

- [ ] Registrar criterios, con nombre único y peso entero positivo.
- [ ] Registrar opciones de los dos tipos, con un puntaje por criterio.
- [ ] Registrar casos de uso, con presupuesto y puntaje mínimo.
- [ ] Evaluación individual, sin duplicar combinaciones.
- [ ] Comparar todas las opciones, reemplazando las evaluaciones del caso.
- [ ] Actualizar el presupuesto y recalcular, sin cambiar fechas.
- [ ] Actualizar precio o consumo y recalcular en todos los casos.
- [ ] Listar criterios, opciones, casos, evaluaciones y ranking.

Código

- [ ] Clases `Sistema`, `OpcionIA`, `ServicioAPI`, `PlanMensual`, `Criterio`, `CasoUso` y `Evaluacion`.
- [ ] `OpcionIA` es abstracta, y ambos tipos implementan `costo_estimado()`.
- [ ] Puntajes como diccionario `{codigo_criterio: puntaje}`.
- [ ] Evaluaciones con referencias a la opción y al caso.
- [ ] Al menos una excepción propia.
- [ ] Fecha y hora registradas con `datetime`.
- [ ] Puntajes mostrados con dos decimales.
- [ ] Menús con los números y textos de la letra.
- [ ] Errores de entrada controlados, sin finalizar el programa.

Entrega

- [ ] Estructura exacta de la pág. 5 de la letra, sin archivos agregados.
- [ ] README con instrucciones y número de equipo.
- [ ] Librerías externas declaradas y justificadas, si se usa alguna.
- [ ] Probado con Python `>= 3.10`.
- [ ] Archivo comprimido enviado por mail hasta el 8 de noviembre a las 23:59.

12 Lista de tareas

Esta lista traduce el plan del capítulo 6 en tareas concretas, en el orden en que conviene hacerlas. Cada tarea nombra el archivo y la función o clase que toca, y termina con algo observable. Marcar una tarea solo cuando ese resultado se vio en pantalla.

12.1 Etapa 0: preparación

- [ ] Crear la carpeta `laboratorio_2026_grupoN/` con `main.py`, `README.md`, `entities/` y `exceptions/`, tal como figura en la letra (pág. 5). Verificar con `python3 --version` que la versión es 3.10 o superior.
- [ ] Leer la letra completa una vez más y copiar en un papel las fórmulas de costo, la fórmula de puntaje y el orden de los tres estados. Se consultan en cada etapa.
- [ ] Acordar en el equipo los nombres de los métodos de `Sistema` (sección 3.6) antes de escribir código. Son el contrato entre el dominio y la consola.

12.2 Etapa 1: clases del diagrama y excepciones

- [ ] `exceptions/ErrorLaboratorio.py`: escribir `class ErrorLaboratorio(Exception): pass`.
- [ ] `entities/Criterio.py`: constructor con `codigo`, `nombre` y `peso`, los tres como atributos privados con `@property` de lectura.
- [ ] `entities/CasoUso.py`: constructor con `codigo`, `nombre`, `presupuesto` y `puntaje_minimo`. El presupuesto lleva setter porque la letra pide actualizarlo.
- [ ] `entities/OpcionIA.py`: clase abstracta `OpcionIA(ABC)` con `codigo`, `nombre`, `puntajes` y `costo_estimado()` marcado con `@abstractmethod`.
- [ ] `entities/OpcionIA.py`: `ServicioAPI` con `costo_por_1000` y `consultas_estimadas`; `costo_estimado()` devuelve `consultas / 1000 * costo_por_1000`.
- [ ] `entities/OpcionIA.py`: `PlanMensual` con `usuarios` y `costo_por_usuario`; `costo_estimado()` devuelve `usuarios * costo_por_usuario`.
- [ ] `entities/OpcionIA.py`: en ambas derivadas, `tipo()`, `descripcion_costos()` y `actualizar_costos(...)`, que se usan en los listados y en la actualización.
- [ ] `entities/Evaluacion.py`: constructor con `codigo`, `opcion`, `caso` y `fecha`; métodos `puntaje_total(criterios)`, `costo_total()` y `recalcular(criterios)` con los tres estados en el orden de la letra.
- [ ] `entities/Sistema.py`: constructor con las cuatro listas privadas y los cuatro contadores de código que empiezan en 1.
- [ ] Comprobar desde el intérprete: `ServicioAPI(1, "X", {}, 40, 5000).costo_estimado()` da `200.0` y `OpcionIA(1, "X", {})` lanza `TypeError`.

12.3 Etapa 2: menú principal y submenús

- [ ] `main.py`: `leer_entero(mensaje, minimo, maximo)`, `leer_flotante(mensaje, minimo, maximo)` y `leer_texto(mensaje)`. Cada una repite hasta recibir un dato válido y nunca deja salir una excepción.
- [ ] `main.py`: `main()` con el menú principal de la letra (opciones 1 a 5) en un `while` que termina solo con 5, y el bloque `if __name__ == "__main__": main()`.
- [ ] `main.py`: `menu_registrar`, `menu_actualizar` y `menu_listar`, cada uno con sus números exactos y la opción Salir que vuelve al menú anterior.
- [ ] Comprobar: recorrer todos los menús, ingresar `abc`, `0` y `9` en cada uno, y ver el mensaje de reintento sin que el programa termine.

12.4 Etapa 3: operaciones de Registrar

- [ ] `Sistema.validar_nombre(registros, nombre)`: rechaza el nombre repetido sin distinguir mayúsculas, lanzando `ErrorLaboratorio`. El nombre vacío lo rechaza antes `leer_texto` en `main.py`: una regla por capa, como en la sección 8.2.
- [ ] `Sistema.registrar_criterio(nombre, peso)` y `main.registrar_criterio(sistema)`: peso entero positivo; bloquear nuevos criterios cuando ya existe una opción.
- [ ] `main.pedir_puntajes(criterios)`: pide un entero de 0 a 10 por cada criterio y arma el diccionario `{codigo_criterio: puntaje}`.
- [ ] `Sistema.registrar_servicio_api(...)`, `Sistema.registrar_plan_mensual(...)` y `main.registrar_opcion(sistema)`: exigir al menos un criterio, pedir nombre, tipo, datos de costo y puntajes, en ese orden.

- [ ] `Sistema.registrar_caso(nombre, presupuesto, puntaje_minimo)` y `main.registrar_caso(sistema)`: presupuesto mayor o igual que cero y mínimo entre 0 y 10.
- [ ] Comprobar: registrar dos criterios, intentar uno repetido con otras mayúsculas, registrar una opción de cada tipo con el ejemplo de la letra (40 USD por 1000 consultas y 5000 consultas) y un caso. Cada registro muestra su código.

12.5 Etapa 4: operaciones de Listar

- [ ] `main.listar_criterios`: código, nombre y peso.
- [ ] `main.listar_opciones`: código, nombre, tipo, datos de costo, costo mensual con dos decimales y puntajes por criterio.
- [ ] `main.listar_casos`: código, nombre, presupuesto, puntaje mínimo y cantidad de evaluaciones recomendadas (cero si no hay).
- [ ] `Sistema.evaluaciones_de_caso(caso)` y `Sistema.ranking_caso(caso)`: se escriben aquí porque `listar_casos` cuenta las recomendadas con `len(sistema.ranking_caso(caso))`. Mientras no haya evaluaciones devuelven una lista vacía; su orden se comprueba en la etapa 6.
- [ ] `main.listar_evaluaciones` y `main.listar_ranking`: dejar el esqueleto que avisa «no hay evaluaciones» o «no hay recomendadas»; se completan en las etapas 5 y 6.
- [ ] Comprobar: la opción API del ejemplo aparece con `USD 200.00` y el caso aparece con 0 recomendadas.

12.6 Etapa 5: evaluación individual

- [ ] `Sistema.evaluar(opcion, caso)`: rechazar la combinación repetida con `ErrorLaboratorio`; si no existe, crear la evaluación con código consecutivo y `datetime.now()`, y recalcular su estado.
- [ ] `main.seleccionar_opcion` y `main.seleccionar_caso`: listar lo registrado y pedir el código; si no hay nada registrado, avisar y volver.
- [ ] `main.registrar_evaluacion(sistema)`: mostrar código, fecha, opción, caso, puntaje total con dos decimales, costo y estado.
- [ ] `main.listar_evaluaciones`, usando `Sistema.evaluaciones_de_caso(caso)`: código, fecha, opción, presupuesto actual del caso, costo, puntaje y estado.
- [ ] Comprobar los tres estados con un mismo caso: una opción cara (fuera de presupuesto), una barata con puntaje bajo (puntaje insuficiente) y una que cumpla ambos (recomendada). Repetir una combinación y ver el aviso de duplicado.

12.7 Etapa 6: comparar opciones y ranking

- [ ] `Sistema.comparar_caso(caso)`: eliminar todas las evaluaciones previas de ese caso, conservar las de otros casos y crear una nueva por cada opción con códigos que no se reutilizan.
- [ ] `main.comparar(sistema)`: si no hay opciones, avisar y volver; si hay, mostrar las evaluaciones creadas.
- [ ] Completar el orden en `Sistema.ranking_caso(caso)`: solo las recomendadas, por puntaje descendente, luego costo ascendente, luego código de opción ascendente.
- [ ] `main.listar_ranking`: posición, opción, puntaje y costo; avisar si no hay recomendadas.
- [ ] Comprobar: evaluar una opción de forma individual, luego comparar, y ver que la evaluación individual desapareció y los códigos siguieron contando. Comprobar un empate de puntaje resuelto por costo.

12.8 Etapa 7: actualizar

- [ ] `Sistema.actualizar_presupuesto(caso, nuevo)` y `main.actualizar_presupuesto(sistema)`: cambiar el presupuesto y recalcular solo los estados de ese caso, sin crear evaluaciones ni cambiar fechas.
- [ ] `Sistema.actualizar_costos_opcion(opcion, primer_valor, segundo_valor)` y `main.actualizar_precio(sistema)`: pedir los dos valores según el tipo de la opción y recalcular costo y estado en todos los casos donde esté evaluada.
- [ ] Comprobar: anotar código y fecha de una evaluación, actualizar el presupuesto y el precio, listar de nuevo y ver que código y fecha no cambiaron y que el estado sí.

12.9 Etapa 8: prueba completa y entrega

- [ ] Ejecutar el guion completo de la sección 10.1, parte A y parte B, y comparar cada número con la tabla.
- [ ] Recorrer el checklist de la sección 11.3 y corregir lo que falle.
- [ ] Completar `README.md`: número de equipo, requisitos, cómo ejecutar y estructura.
- [ ] Renombrar la carpeta con el número de equipo, borrar `__pycache__`, comprimir y descomprimir en otra carpeta para probar que `python3 main.py` arranca.
- [ ] Enviar antes del 8 de noviembre a las 23:59.